

Solution Design Document

Architecture, Distributed RDD Processing Strategy, and Intelligent-Agent Design for
Five Distributed AI Systems Built on PySpark

Common Design Philosophy

All five systems follow the same layered architecture pattern: (1) a distributed ingestion layer that parallelizes raw data into Spark RDDs, (2) one or more transformation stages using core RDD operators (map, filter, groupByKey, reduceByKey, aggregateByKey, join, sortBy) to reduce, join, and score large-scale data in a fault-tolerant and horizontally scalable way, and (3) a lightweight intelligent-agent layer that consumes the reduced/aggregated state to make fast, adaptive, real-time decisions using simple online-learning update rules. This separation allows the heavy, large-scale numeric computation to be handled by Spark's distributed engine, while decision logic remains simple, explainable, and quick to adapt.

Solution 1: Hybrid AI Recommendation Engine

System Architecture

- **Data Ingestion Layer:** raw user-item interaction logs (ratings/clicks) and item metadata are loaded into Spark as RDDs (sc.parallelize / textFile in production, reading from HDFS, Kafka, or S3 at scale).
- **Collaborative Filtering (CF) Module:** groups interactions by user (groupByKey), generates item-item co-occurrence pairs, and aggregates co-rating strength with reduceByKey — an RDD-native equivalent of item-based collaborative filtering.
- **Content-Based Filtering (CBF) Module:** maps each item's metadata (genre/category/tags) to a similarity score against the target user's preference profile.
- **Fusion Layer:** normalizes CF strength and CBF scores independently, then joins the two RDDs on item_id and computes a weighted hybrid score (score = 0.6*CF + 0.4*CBF, tunable).
- **Intelligent Agent Layer:** a RecommenderAgent observes real-time engagement feedback (click-through / dwell-time signals) and adapts its confidence weight online, which can be fed back to re-tune the CF/CBF blend ratio without a full model retrain.

Key PySpark RDD Operations Used

- map() — transform raw rows into (key, value) pairs for downstream aggregation.
- groupByKey() — group interactions by user id to compute per-user item co-occurrence.
- flatMap() — expand each user's item list into pairwise (item_a, item_b) co-rating records.
- reduceByKey() — aggregate co-rating strength per item pair across the whole distributed dataset.
- join() — combine normalized collaborative-filtering scores with content-based scores by item id.
- sortBy() — rank fused hybrid scores to produce the final Top-N recommendation list.

Role of Intelligent Agents

The RecommenderAgent implements an online learning rule: $w_{t+1} = w_t + \alpha * (\text{feedback}_t - w_t)$. Positive engagement (click, high rating, long dwell time) nudges the agent's confidence upward; negative feedback (skip, low rating) nudges it downward. This adaptive weight can dynamically bias the hybrid fusion formula toward collaborative signals (when the agent is confident enough peer data exists) or toward content-based signals (for cold-start users/items where collaborative data is sparse).

Representative RDD Code Snippet

```
user_items = ratings_rdd.map(lambda x: (x[0], (x[1], x[2]))).groupByKey()
item_similarity = user_items.flatMap(item_pairs).reduceByKey(lambda a, b: a+b)
hybrid = cf_norm.join(content_scores)\
    .mapValues(lambda cv: 0.6*cv[0] + 0.4*cv[1])\
    .sortBy(lambda kv: -kv[1])
```

Solution 2: Smart Disaster Detection System

System Architecture

- Multi-Source Ingestion Layer: three independent RDD streams are created for ground/seismic sensor telemetry, satellite-derived imagery anomaly indices, and social-media disaster-keyword mention counts — each keyed by geographic zone.
- Per-Source Aggregation: each stream is reduced to a per-zone average/sum using `mapValues + reduceByKey`, converting raw high-frequency readings into zone-level statistics.
- Data Fusion Engine: the three aggregated RDDs are joined on zone id and combined via a weighted-sum fusion function ($0.50 * \text{sensor_severity} + 0.35 * \text{satellite_anomaly} + 0.15 * \text{social_signal}$) to compute a single unified Disaster Risk Index per zone.
- Intelligent Agent Coordination Layer: a `DisasterCoordinatorAgent` evaluates each zone's fused risk score against an adaptive threshold and classifies zones as NORMAL / WATCH / CRITICAL, triggering early-warning alerts and response-team dispatch recommendations for critical zones.

Key PySpark RDD Operations Used

- `mapValues()` — apply running-average accumulation to each source stream independently.
- `reduceByKey()` — aggregate multiple readings per zone into one statistic per source.
- `join()` (chained) — fuse sensor, satellite and social RDDs together by zone key.
- `mapValues()` — apply the weighted fusion formula to compute the unified risk index.
- `sortBy()` / `filter()` — rank zones by risk and isolate those crossing the alert threshold.

Role of Intelligent Agents

The coordinator agent adapts its alert sensitivity threshold based on recent false-alarm / confirmed-event feedback ($\text{threshold} = 0.5 + 0.1 * (1 - \text{weight})$); a history of accurate alerts lowers the required threshold (faster response), while a history of false alarms raises it (fewer nuisance alerts). In a full deployment, multiple coordinator agents (one per regional cluster) would communicate to avoid duplicate dispatch and to prioritize response resources across simultaneously affected zones.

Representative RDD Code Snippet

```
fused = avg_sensor.join(avg_satellite).join(total_social)\
.mapValues(lambda v: 0.5*(v[0][0]/10) + 0.35*v[0][1] + 0.15*min(v[1]/50,1))\
.sortBy(lambda kv: -kv[1])
status = 'CRITICAL' if risk_score >= adaptive_threshold else 'WATCH'
```

Solution 3: AI-Based Fraud Detection Framework

System Architecture

- Distributed Ingestion Layer: transaction records (tx_id, account, amount, hour) are loaded as an RDD, scalable to millions of records per batch/micro-batch across a Spark cluster.
- Behavioral Profiling Module: per-account statistics (mean, standard deviation of transaction amount) are computed in a single distributed pass using aggregateByKey, avoiding the need to shuffle raw records more than once.
- Anomaly Scoring Engine: each transaction is scored using a broadcast variable containing per-account statistics, combining a normalized z-score deviation with rule-based signals (e.g., odd-hour activity) into a single anomaly score.
- Alert Ranking Layer: suspicious transactions are filtered (score >= cutoff) and sorted in a fully distributed manner to surface the highest-risk cases first.
- Collaborative Agent Layer: a FraudDetectorAgent and a HumanReviewFeedbackAgent share a feedback loop — analyst-confirmed outcomes continuously retune the alert cutoff.

Key PySpark RDD Operations Used

- aggregateByKey() — compute (sum, sum-of-squares, count) per account in one distributed pass to derive mean and standard deviation without repeated shuffles.
- broadcast() — efficiently distribute the small per-account statistics table to every worker node for fast, local anomaly scoring (avoids a costly join for a lookup-only dataset).
- map() — apply the anomaly-scoring function to every transaction in parallel.
- filter() + sortBy() — isolate and rank transactions exceeding the suspicion threshold.

Role of Intelligent Agents

Two agents collaborate: the FraudDetectorAgent proposes ALERT/MONITOR decisions from the anomaly score, while the HumanReviewFeedbackAgent absorbs analyst verdicts (confirmed fraud vs. false positive) and adapts a shared confidence weight. Over successive review cycles the alert cutoff tightens or loosens automatically, improving precision without manual rule rewriting — a lightweight, interpretable alternative to full model retraining.

Representative RDD Code Snippet

```
acc_stats = acc_amounts.aggregateByKey((0.0,0.0,0),
lambda agg,v: (agg[0]+v, agg[1]+v*v, agg[2]+1),
lambda a,b: (a[0]+b[0], a[1]+b[1], a[2]+b[2])).collectAsMap()
stats_bc = sc.broadcast(acc_stats)
scored = tx_rdd.map(score_tx).filter(lambda t: t[4] >= 0.5).sortBy(lambda t: -t[4])
```

Solution 4: Autonomous Supply Chain Optimization System

System Architecture

- **Distributed Data Layer:** inventory levels and demand forecasts for every (warehouse, product) pair are represented as keyed RDDs, enabling processing at the scale of thousands of SKUs across hundreds of warehouses.
- **Gap Analysis Engine:** an RDD join between inventory and forecasted demand computes a shortage/surplus gap for every warehouse-product pair in a single distributed operation.
- **Network Aggregation:** shortages are summarized per product (reduceByKey) to identify the most critical items, and surpluses are summarized per warehouse to identify redistribution sources.
- **Decentralized Agent Layer:** one WarehouseAgent per warehouse autonomously proposes stock transfers, balancing transfer urgency against per-unit logistics cost — decisions are made locally/in parallel rather than through a single central optimizer.

Key PySpark RDD Operations Used

- `map()` — reshape inventory and demand records into ((warehouse, product), value) key-value pairs.
- `join()` — align inventory with forecasted demand per warehouse-product key.
- `mapValues()` — compute the shortage/surplus gap for each pair.
- `filter()` — split the gap RDD into shortage-only and surplus-only partitions.
- `reduceByKey()` — aggregate shortages by product and surpluses by warehouse across the network.

Role of Intelligent Agents

Each WarehouseAgent maintains its own adaptive weight, tuned from historical on-time-delivery feedback for that warehouse. When proposing a transfer, the agent blends cost efficiency ($1 - \text{cost_per_unit}/10$) with shortage urgency ($\text{units_needed}/300$) using its personal confidence weight, producing a decentralized transfer-confidence score. Because each agent acts independently on locally relevant data, the design avoids a single central bottleneck and scales naturally as more warehouses are added to the network.

Representative RDD Code Snippet

```
gap_rdd = inv_rdd.join(dem_rdd).mapValues(lambda v: v[1]-v[0])
shortage_by_product = gap_rdd.filter(lambda kv: kv[1]>0)\
    .map(lambda kv: (kv[0][1], kv[1])).reduceByKey(lambda a,b: a+b)
confidence = agent.weight*(1-cost/10) + (1-agent.weight)*urgency
```

Solution 5: Personalized Learning Analytics Platform

System Architecture

- Interaction Ingestion Layer: per-attempt learner interaction records (student id, topic, score, time spent) are ingested as an RDD, scaling to thousands of concurrent learners and millions of interaction events.
- Student Performance Aggregator: per-student average performance is computed via a single distributed map + reduceByKey pass over all interactions.
- Topic Difficulty Analyzer: per-topic average score and average time-on-task are computed in parallel, surfacing which topics are systematically harder across the learner population.
- At-Risk Detection: a distributed filter isolates learners whose rolling average performance falls below a mastery threshold, enabling proactive educator intervention.
- Adaptive Tutor Agent Layer: a TutorAgent consumes each learner's aggregated performance and recent engagement feedback to recommend the next best content difficulty level in real time.

Key PySpark RDD Operations Used

- map() — convert raw interaction tuples into (student_id, (score, 1)) / (topic, (score, time, 1)) pairs ready for aggregation.
- reduceByKey() — sum scores/counts per student and per topic in a single distributed pass.
- mapValues() — convert summed accumulators into final averages.
- filter() — isolate at-risk students below the mastery threshold.
- mean() (RDD action) — compute the class-wide average performance across the whole distributed dataset.

Role of Intelligent Agents

The AdaptiveTutorAgent adapts a 'difficulty bias' weight from a stream of engagement signals (lesson completed vs. dropped). Combined with a learner's aggregated average score, the agent recommends remedial, standard-pace, or advanced/enrichment content — personalizing pacing at the individual level while the underlying analytics (topic difficulty, at-risk detection) remain computed at population scale using distributed RDD operations.

Representative RDD Code Snippet

```
student_perf = inter_rdd.map(lambda x:(x[0],(x[2],1)))\
    .reduceByKey(lambda a,b:(a[0]+b[0], a[1]+b[1])).mapValues(lambda v: v[0]/v[1])
at_risk = student_perf.filter(lambda kv: kv[1] < 50).count()
recommendation = tutor.recommend_next(avg_score)
```